

VITyarthi - Build Your Own Project

TripMate

A Student-Friendly College Travel Planner

Student Name	Kriti Gothwal
Registration Number	<u>26MIM10023</u>
Course / Subject	Introduction to Problem Solving and Programming
Program	Integrated M.Tech (AI)
Submission Date	26/09/2026

1. Introduction

Every semester, my friends and I end up planning some small trip - sometimes a college trip, sometimes just a weekend getaway. And almost every single time, the "planning" is just a messy WhatsApp group chat with half the information missing - nobody knows the exact budget, nobody has a proper day plan, and someone always forgets to carry something important. While learning Python this semester, I thought this was a perfect real-life problem I could try to solve with the concepts we have been taught - loops, functions, dictionaries and file handling. That is how the idea for **TripMate** came about.

TripMate is a menu-driven Python console application that helps a student plan a college trip end-to-end: choosing a destination, estimating a budget, comparing transport, generating a day-wise itinerary, and remembering what to pack, and finally saving the trip for later.

2. Problem Statement

College students plan trips very unorganized - budgets are guessed roughly, itineraries are not written down anywhere, and packing lists are made (if at all) five minutes before leaving. There is no simple offline tool built specifically for a student's small trip that brings destination info, budget calculation, itinerary planning and packing checklist together in one place. TripMate is built to solve exactly this gap using core Python programming concepts, without needing internet access, logins, or any paid service.

3. Functional Requirements

The project provides the following major functional modules and features:

- **Trip Planning Module** - collects starting location, destination, number of days, number of people and trip type from the user, with input validation.
- **Destination Search Module** - lets the user search a built-in destination database by name or by trip type (beach/hills/city/religious).
- **Budget Calculator Module** - calculates an estimated budget (transport + hotel + food + activities) based on user's choice of transport and number of days/people.
- **Transport Comparison Module** - compares Bus, Train, Flight and Cab on cost, time and comfort, and automatically points out the cheapest and fastest option.
- **Itinerary Generator Module** - builds a day-wise schedule using the destination's popular activities from the database.
- **Packing Checklist Module** - shows a checklist of items depending on the type of trip selected.
- **Save/Load Module (File Handling)** - saves a planned trip (with its budget) into a CSV file and can list all previously saved trips back to the user.

- **Emergency Contacts Module** - displays useful general helpline numbers and lets the user note one personal emergency contact for the trip.

Input / Output structure: All input is taken through the console using `input()`, validated where needed (e.g. days and people must be positive numbers), and all output is displayed back on the console in a clearly formatted way (tables, checklists, day-wise schedule).

Workflow: The user is shown a numbered menu (1-10) in a loop. Based on the chosen number, control goes to the matching module/function. The user can repeat this any number of times until they choose to exit.

4. Non-Functional Requirements

Requirement	How it is handled in TripMate
Performance	All data is kept in memory (dictionaries/lists) so the program responds instantly; no network calls are made.
Usability	Simple numbered menu, clear prompts, and formatted output (tables/checklists) so it is easy for any student to use without instructions.
Reliability	The program keeps running in a loop and does not crash on wrong input - it shows a message and asks again instead.
Maintainability	Code is split into 9 separate files by feature, so any one feature can be updated without touching the others.
Error Handling	Invalid menu choices, invalid number inputs, and file read/write errors are all caught and shown as friendly messages (using if-checks and try/except).
Scalability	New destinations can be added to <code>destinations_data.py</code> , and new menu options can be added to <code>main.py</code> , without changing the rest of the program.
Resource Efficiency	Uses only Python's built-in <code>os</code> and <code>csv</code> modules - no heavy external libraries are required to run the project.

5. System Architecture

TripMate follows a simple layered structure even though it is a console application. The **Presentation Layer** (`main.py`) is responsible for showing the menu and taking user input. The **Business Logic Layer** contains the individual feature modules (trip planning, budget calculation, itinerary, transport comparison, packing checklist, emergency contacts). The **Data Layer** is made up of the in-memory destination dictionary (`destinations_data.py`) and the file-based storage (`file_storage.py` + `saved_trips.csv`).

TripMate - System Architecture Diagram

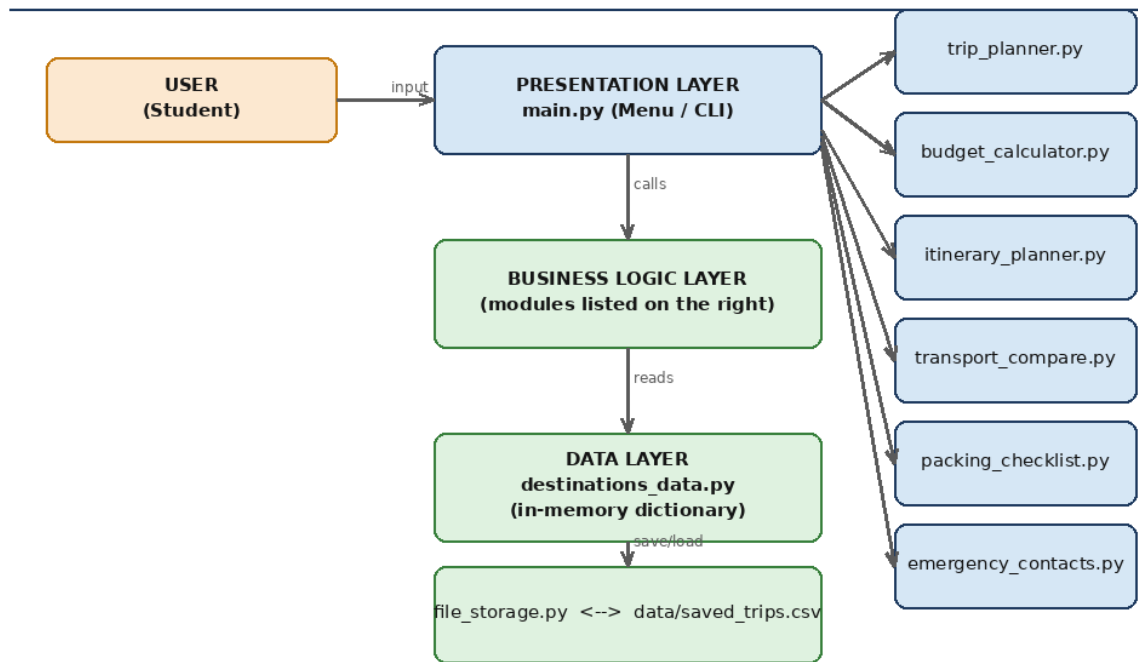


Figure 1: System Architecture Diagram

6. Design Diagrams

6.1 Use Case Diagram

This diagram shows all the actions a student (the only actor in this system) can perform using TripMate.

TripMate - Use Case Diagram

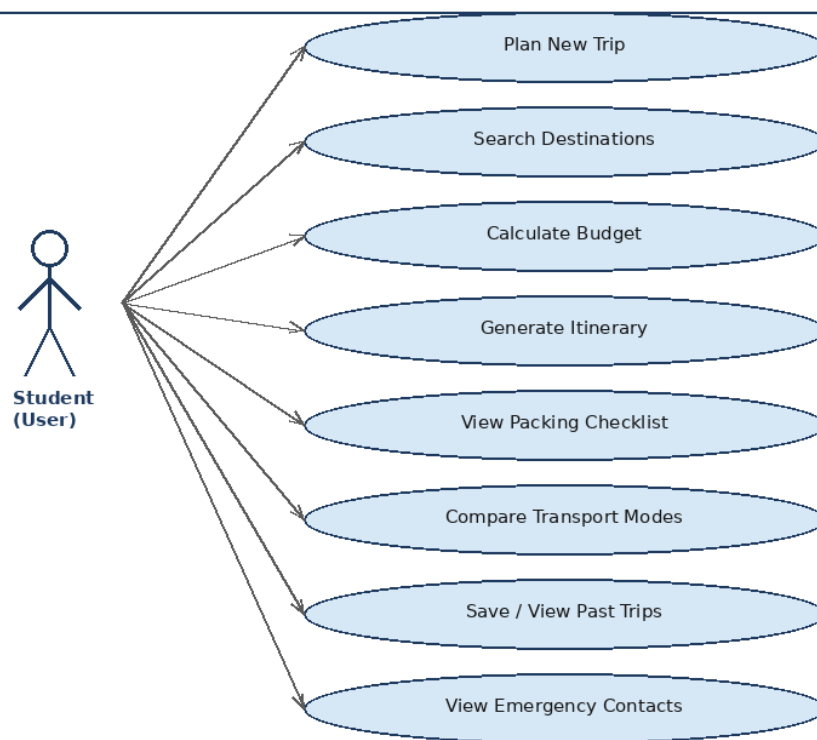


Figure 2: Use Case Diagram

6.2 Workflow / Process Flow Diagram

This shows the overall flow of the program from start to exit, including the menu loop.

TripMate - Process Flow / Workflow Diagram

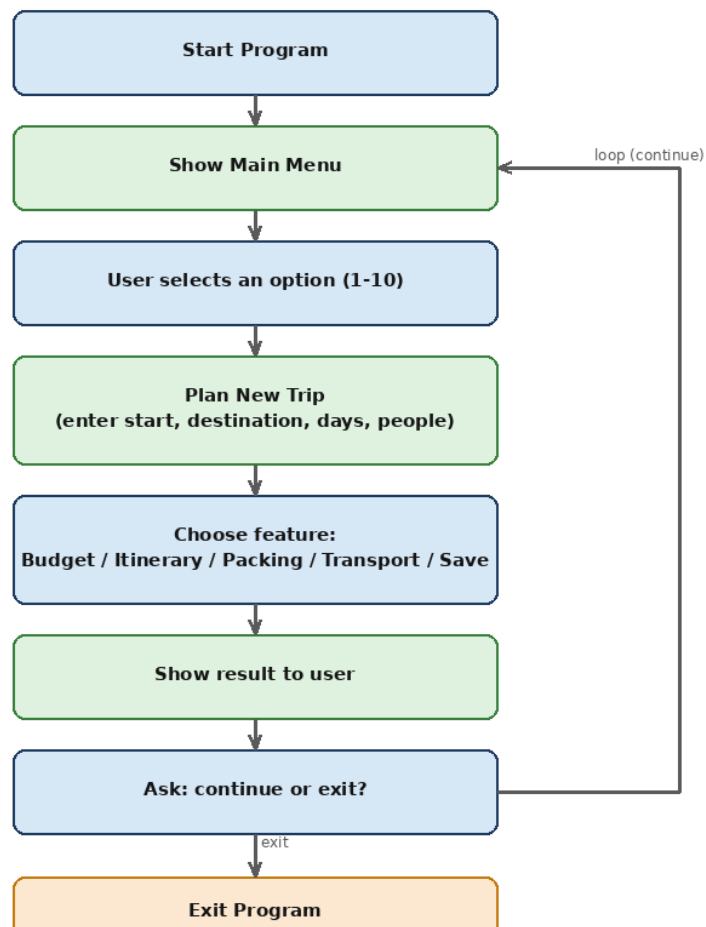


Figure 3: Process Flow / Workflow Diagram

6.3 Sequence Diagram

This shows the sequence of calls between the modules for one common flow: planning a trip, checking its budget, and then saving it.

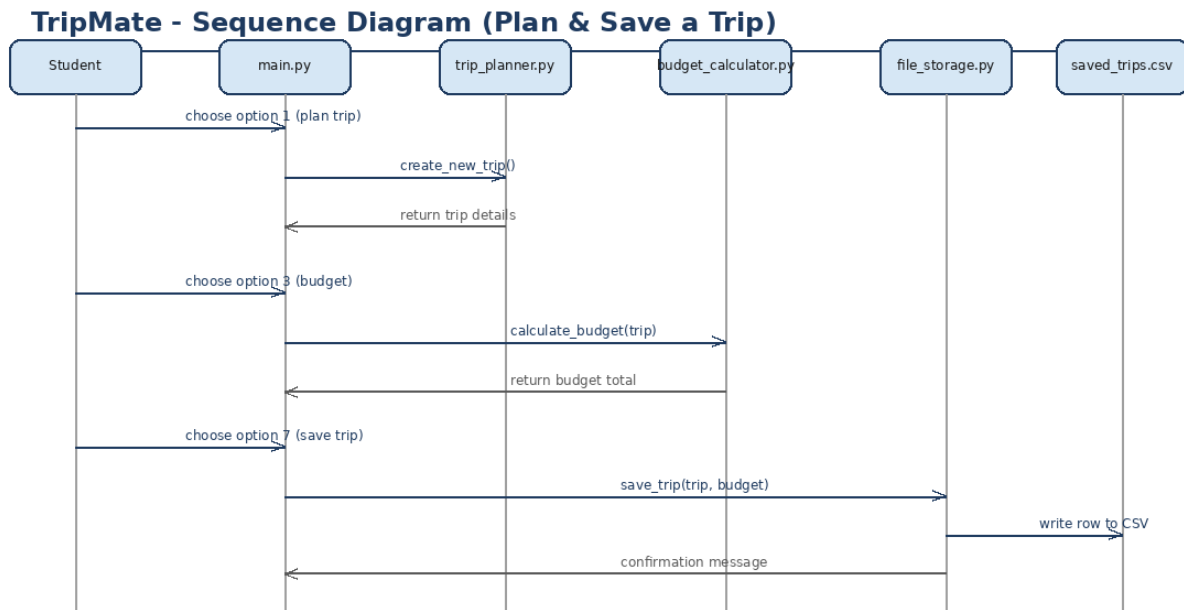


Figure 4: Sequence Diagram (Plan and Save a Trip)

6.4 Component Diagram

Since this project is written using functions rather than classes (as per what we have learned so far in our Python course), a **Component/Module diagram** is shown instead of a class diagram - it shows the 9 Python files and the important functions inside each, and how they call each other.

TripMate - Component Diagram (Module Level)

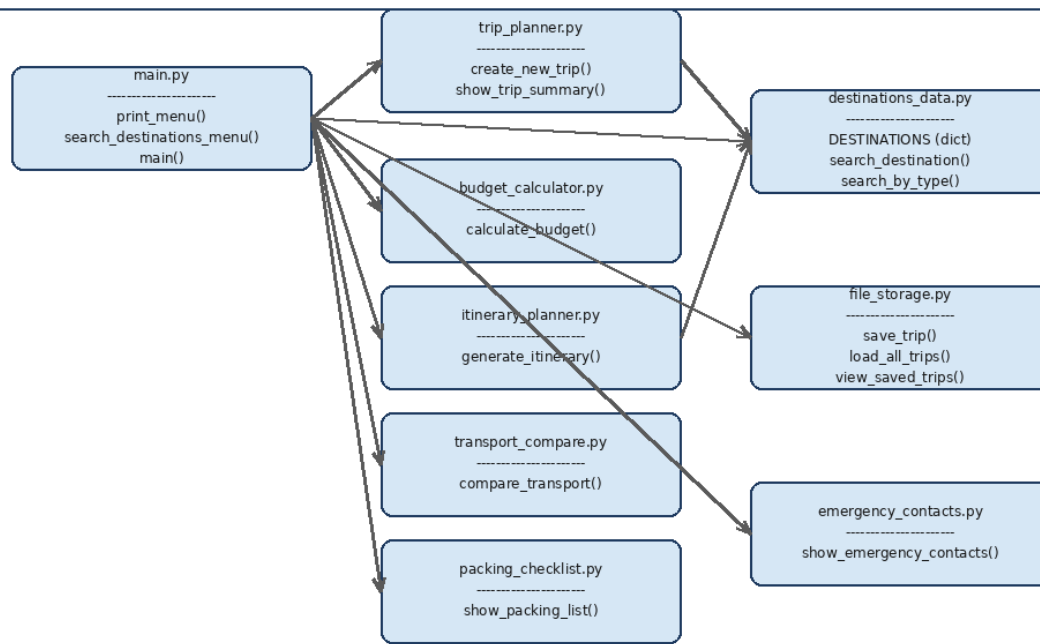


Figure 5: Component Diagram (Module Level)

6.5 Data Storage Design

TripMate does not use a relational database (no SQL is used in this project), so a formal ER diagram does not apply here. Instead, saved trips are stored as rows in a single CSV file, which is shown below like a simple table/schema.

TripMate - Data Storage Structure (saved_trips.csv)

TripMate does not use a relational database. Trip records are stored as rows in a single CSV file: data/saved_trips.csv. Each row = one saved trip.

start	destination	days	people	trip_type	total_budget
Delhi	Manali	3	2	college	10000

Field notes:

- start / destination : text (city names)
- days / people : whole numbers
- trip_type : text (college / trek / beach / city / religious)
- total_budget : number, calculated by budget_calculator.py

Figure 6: Data Storage Structure (saved_trips.csv)

7. Design Decisions & Rationale

- **Dictionaries over classes:** A trip is represented as a plain Python dictionary instead of a class, since dictionaries were more comfortable for me at this stage and are enough for this project's needs.
- **CSV file instead of a database:** A CSV file was chosen for saving trips because it directly uses the file handling concepts taught in class, and does not need any extra setup like installing a database.
- **Splitting into 9 files:** Each feature was put in its own file so the code stays organized and it is easier to test and fix one feature without breaking the others.
- **Menu-driven design:** A simple numbered menu was chosen (instead of typed commands) since it is the easiest and most beginner-friendly way to let the user pick a feature.
- **Built-in destination database:** Since the project must work fully offline (no internet/API), a small dictionary of common Indian destinations was created manually instead of calling any travel API.

8. Implementation Details

The project is implemented in Python 3, split across the following files inside the **src/** folder:

File	Responsibility
main.py	Program entry point, shows the menu and connects all other modules together
trip_planner.py	Takes trip details (start, destination, days, people, type) from the user
budget_calculator.py	Calculates estimated budget based on transport, hotel, food and activities
transport_compare.py	Compares Bus/Train/Flight/Cab on cost, time and comfort
itinerary_planner.py	Builds a simple day-wise itinerary using destination activities
packing_checklist.py	Shows a packing checklist based on trip type
destinations_data.py	Holds the built-in dictionary of destinations and search functions
file_storage.py	Saves/loads trips to and from data/saved_trips.csv using the csv module
emergency_contacts.py	Displays emergency helpline numbers

Sample snippet from **budget_calculator.py** showing how the budget is calculated:

```
rooms_needed = (people + 1) // 2
hotel_cost = HOTEL_RATE_PER_DAY * rooms_needed * days
food_cost = FOOD_RATE_PER_DAY * people * days
activity_cost = ACTIVITY_RATE_PER_DAY * people * days
total = transport_cost + hotel_cost + food_cost + activity_cost
```

9. Screenshots / Results

(This section is meant to be filled with actual screenshots taken while running the program on your own machine before final submission - for example the main menu, a sample budget calculation, and the itinerary output. A sample console run is shown below as text since screenshots could not be captured in this environment.)

```
=====
TRIPMATE - COLLEGE TRAVEL PLANNER
=====
1. Plan a New Trip
2. Search Destinations
3. Calculate Travel Budget
...
Enter your choice (1-10): 1
Enter your starting location: Delhi
Enter destination: Manali
Number of days for the trip: 3
Number of people travelling: 2
What kind of trip is this?: college

Great! Here is a summary of your trip:
-----
From : Delhi
Destination : Manali
Days : 3
People : 2
Trip type : college
-----
```

10. Testing Approach

Since this is a console-based project, testing was done manually by running the program and checking the output for different cases:

- Checked that valid destinations (like Manali, Goa, Jaipur) return the correct information.
- Checked that an unknown destination name shows a proper "not found" message instead of crashing.
- Entered negative numbers and text (instead of numbers) for days/people to confirm the input validation loop keeps asking again.
- Entered an invalid menu option (like 0 or 15) to confirm the program shows an error and shows the menu again.
- Saved multiple trips and used 'View Saved Trips' to confirm all of them were read back correctly from the CSV file.
- Ran the full flow start to end (plan trip -> calculate budget -> see itinerary -> packing list -> save -> view saved -> exit) to make sure nothing breaks in between.

11. Challenges Faced

- Deciding how to store a 'trip' - initially I was going to use separate variables for everything, but that became messy, so I switched to using a dictionary for each trip.
- Handling the CSV file correctly - making sure the header row is written only once, and not every time a trip is saved.
- Making sure the program does not crash when the user enters the wrong type of input (like typing 'three' instead of 3) - solved this using `isdigit()` checks in a loop.
- Keeping the itinerary logic simple while still using the real activities of a destination, without writing separate code for every place.

12. Learnings & Key Takeaways

- Got much more comfortable with dictionaries and nested dictionaries after using them for the whole destination database.
- Understood why splitting code into multiple files/modules makes a program easier to manage, instead of writing everything in one huge file.
- Learned practical file handling - reading and writing CSV files using Python's built-in `csv` module.
- Realised how important input validation is - without it, the program would crash very easily on wrong input.
- Got a better idea of how a 'real' application is planned first (requirements, diagrams) before writing any code.

13. Future Enhancements

- Add a graphical user interface (GUI) using Tkinter instead of a console menu.
- Connect to a real travel/weather API to get live prices and weather forecasts for the destination.
- Allow multiple users with separate saved trip histories (basic login system).
- Add a proper database (like SQLite) instead of a CSV file once we learn databases in later semesters.
- Add a feature to split the budget among the group members automatically.

14. References

- Python official documentation - <https://docs.python.org/3/>
- Python csv module documentation - <https://docs.python.org/3/library/csv.html>
- Class notes and lab exercises from the Python Programming course.
- General trip cost/time estimates based on common knowledge of Indian travel options (approximate, for academic demonstration only).